

BACCALAURÉAT

SESSION 2024

Épreuve de l'enseignement de spécialité

NUMÉRIQUE et SCIENCES INFORMATIQUES

Partie pratique

Classe Terminale de la voie générale

Sujet n°02

DURÉE DE L'ÉPREUVE : 1 heure

Le sujet comporte 4 pages numérotées de 1 / 4 à 4 / 4
Dès que le sujet vous est remis, assurez-vous qu'il est complet.

Le candidat doit traiter les 2 exercices.

EXERCICE 1 (10 points)

On considère des chaînes de caractères contenant uniquement des majuscules et des caractères `*` appelées *mots à trous*.

Par exemple `INFO*MA*IQUE`, `***I***E**` et `*S*` sont des mots à trous.

Programmer une fonction `correspond` :

- qui prend en paramètres deux chaînes de caractères `mot` et `mot_a_trous` où `mot_a_trous` est un mot à trous comme indiqué ci-dessus ;
- et qui renvoie :
 - `True` si on peut obtenir `mot` en remplaçant convenablement les caractères `'*'` de `mot_a_trous` ;
 - `False` sinon.

Exemple :

```
>>> correspond('INFORMATIQUE', 'INFO*MA*IQUE')
True
>>> correspond('AUTOMATIQUE', 'INFO*MA*IQUE')
False
>>> correspond('STOP', 'S*')
False
>>> correspond('AUTO', '*UT*')
True
```

EXERCICE 2 (10 points)

On considère au plus 26 personnes A, B, C, D, E, F ... qui peuvent s'envoyer des messages avec deux règles à respecter :

- chaque personne ne peut envoyer des messages qu'à une seule personne (éventuellement elle-même),
- chaque personne ne peut recevoir des messages qu'en provenance d'une seule personne (éventuellement elle-même).

Voici un exemple - avec 6 personnes - de « plan d'envoi des messages » qui respecte les règles ci-dessus, puisque chaque personne est présente une seule fois dans chaque colonne :

- A envoie ses messages à E
- E envoie ses messages à B
- B envoie ses messages à F
- F envoie ses messages à A
- C envoie ses messages à D
- D envoie ses messages à C

Le dictionnaire correspondant à ce plan d'envoi est alors le suivant :

```
plan_a = {'A':'E', 'B':'F', 'C':'D', 'D':'C', 'E':'B', 'F':'A'}
```

Un cycle est une suite de personnes dans laquelle la dernière est la même que la première.

Sur le plan d'envoi `plan_a` des messages ci-dessus, il y a deux cycles distincts : un premier cycle avec A, E, B, F et un second cycle avec C et D.

En revanche, le plan d'envoi `plan_b` ci-dessous :

```
plan_b = {'A':'C', 'B':'F', 'C':'E', 'D':'A', 'E':'B', 'F':'D'}
```

comporte un unique cycle : A, C, E, B, F, D. Dans ce cas, lorsqu'un plan d'envoi comporte un *unique cycle*, on dit que le plan d'envoi est *cyclique*.

Pour savoir si un plan d'envoi de messages comportant N personnes est cyclique, on peut utiliser l'algorithme ci-dessous :

- on part d'un expéditeur (ici A) et on inspecte son destinataire dans le plan d'envoi,
- chaque destinataire devient à son tour expéditeur, selon le plan d'envoi, tant qu'on ne « retombe » pas sur l'expéditeur initial,
- le plan d'envoi est cyclique si on l'a parcouru en entier.

Compléter la fonction `est_cyclique` située à la page suivante en respectant la spécification. On rappelle que la fonction Python `len` permet d'obtenir la longueur d'un dictionnaire.

```

def est_cyclique(plan):
    '''Prend en paramètre un dictionnaire `plan` correspondant à
    un plan d'envoi de messages (ici entre les personnes A, B, C,
    D, E, F).
    Renvoie True si le plan d'envoi de messages est cyclique et
    False sinon.'''
    expéditeur = 'A'
    destinataire = plan[...]
    nb_destinataires = 1

    while destinataire != expéditeur:
        destinataire = ...
        nb_destinataires = ...

    return nb_destinataires == ...

```

Exemples:

```

>>> est_cyclique({'A':'E','F':'A','C':'D','E':'B','B':'F','D':'C'})
False
>>> est_cyclique({'A':'E','F':'C','C':'D','E':'B','B':'F','D':'A'})
True
>>> est_cyclique({'A':'B','F':'C','C':'D','E':'A','B':'F','D':'E'})
True
>>> est_cyclique({'A':'B','F':'A','C':'D','E':'C','B':'F','D':'E'})
False

```